

Unit-IV: ARM Processor

❖ Fundamentals of ARM Processor

The ARM Processor is one of the most widely used processor architectures in embedded systems, mobile devices, and IoT due to its **high performance and low power consumption**.

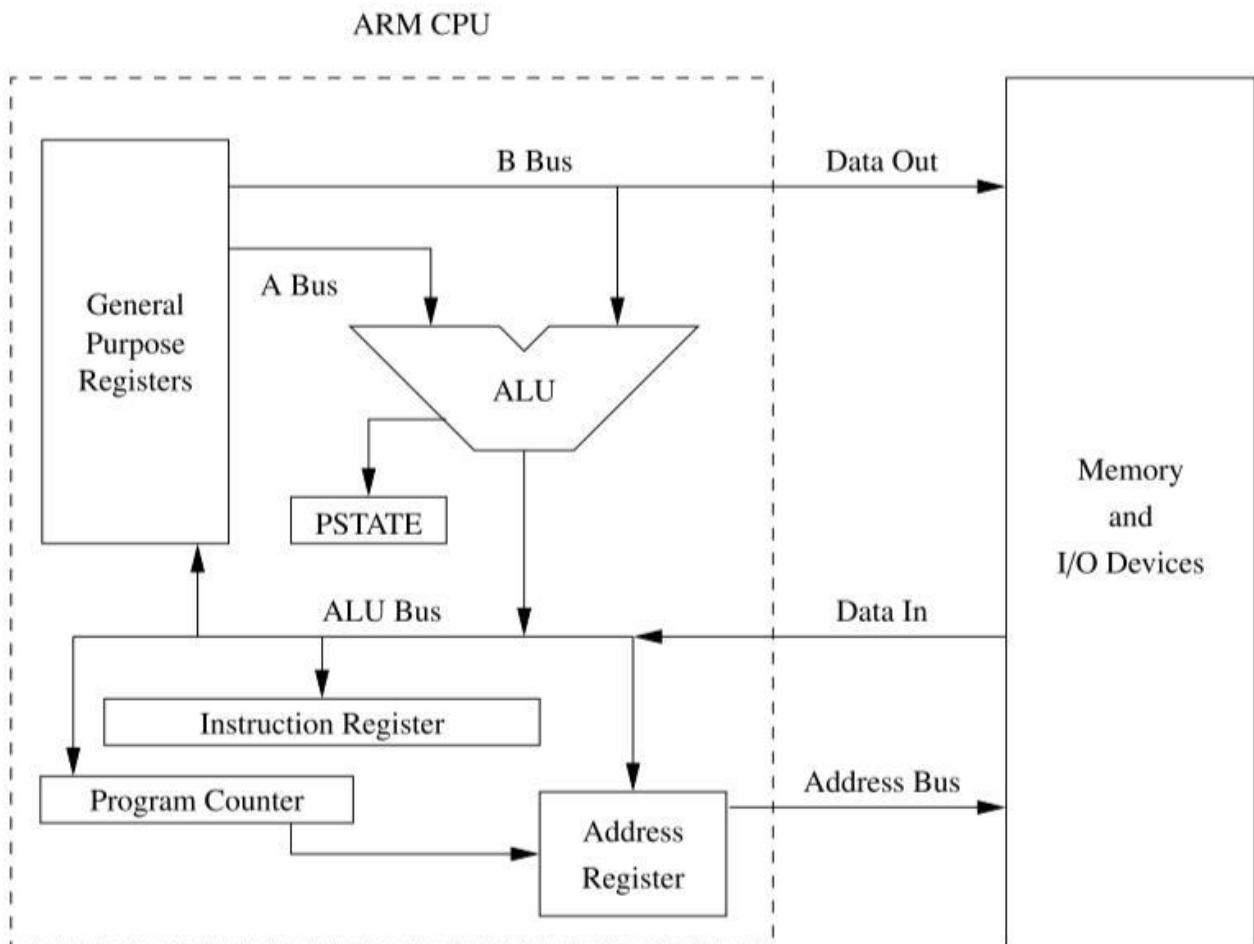
What is ARM?

- **ARM** stands for **Advanced RISC Machine**
- Based on **RISC (Reduced Instruction Set Computer)** architecture
- Designed for **efficiency, speed, and low power usage**

👉 Widely used in:

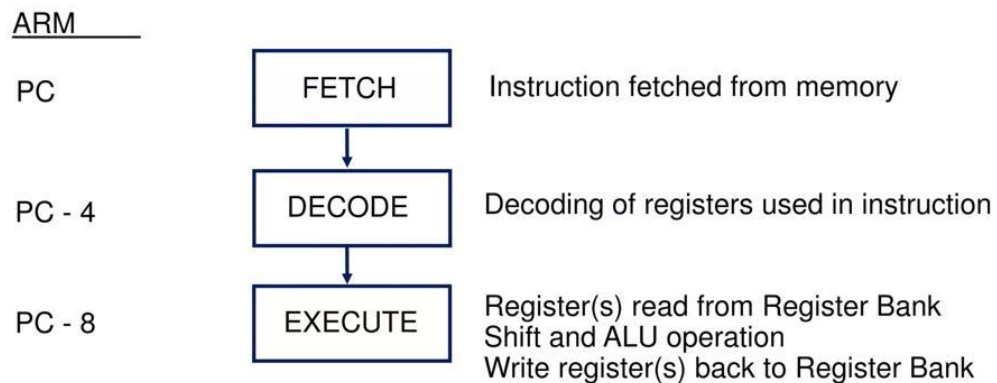
- Smartphones
- Embedded systems
- Tablets
- IoT devices

2. Key Features of ARM Processor

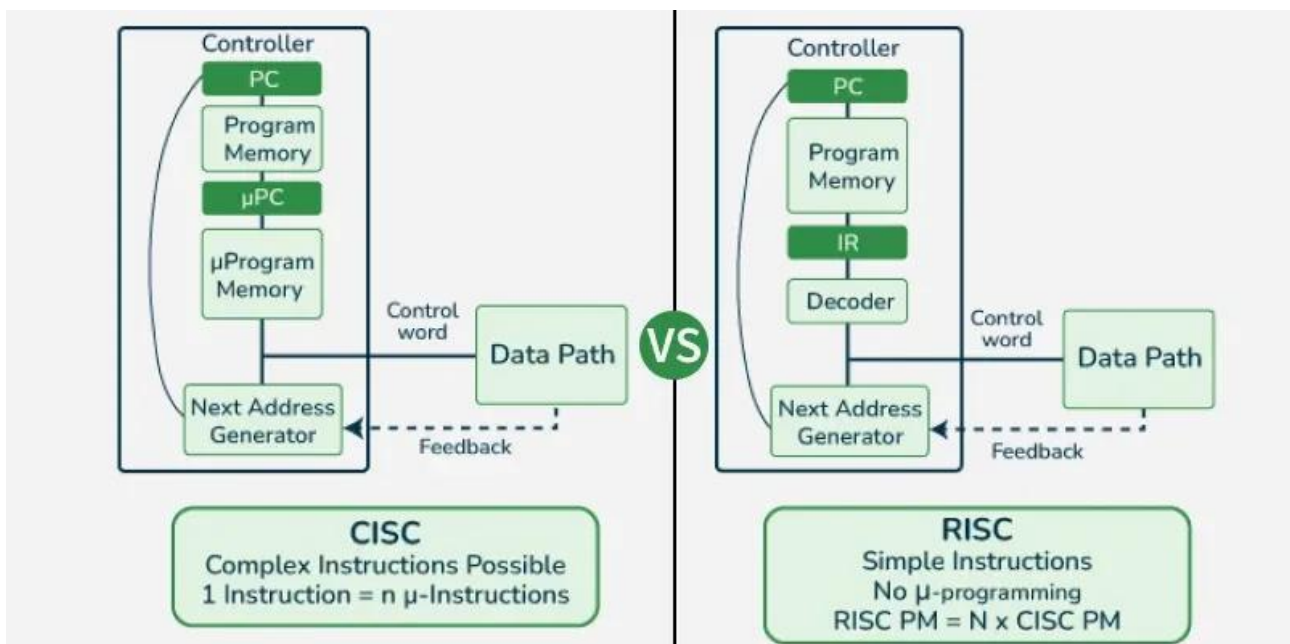


The Instruction Pipeline

- The ARM uses a pipeline in order to increase the speed of the flow of instructions to the processor.
 - Allows several operations to be undertaken simultaneously, rather than serially.



- Rather than pointing to the instruction being executed, the PC points to the instruction being fetched.



◆ RISC Architecture

- Simple instructions
- Fixed instruction length (mostly 32-bit)
- Faster execution

◆ Load/Store Architecture

- Operations performed only on registers
- Memory accessed only via **load/store instructions**

◆ Large Register Set

- Typically **16 general-purpose registers**
- Faster data access

◆ Pipelining

- Multiple instructions executed simultaneously in stages
- Improves performance

◆ Low Power Consumption

- Ideal for battery-operated devices

3. ARM Processor Architecture

Main Components:

- **ALU (Arithmetic Logic Unit)** → Performs operations
- **Register File** → Stores data
- **Control Unit** → Controls execution
- **Bus System** → Transfers data
- **Pipeline Unit** → Improves speed

4. Register Organization in ARM

ARM typically has **16 registers (R0–R15)**:

Register Function

R0–R12 General-purpose registers

R13 Stack Pointer (SP)

R14 Link Register (LR)

R15 Program Counter (PC)

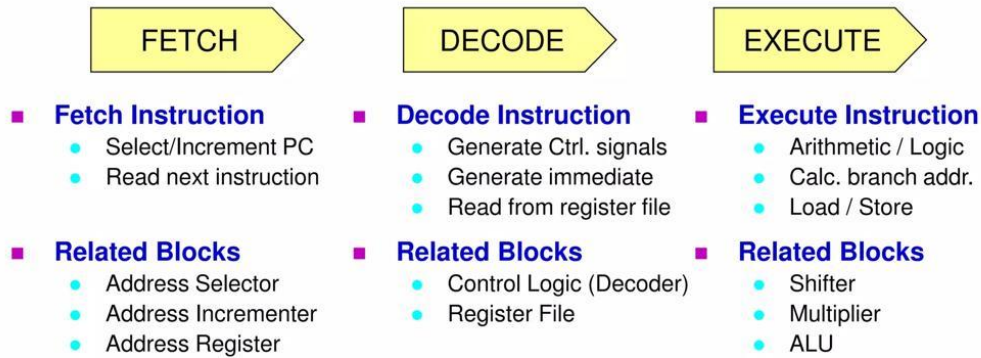
👉 CPSR (Current Program Status Register) stores:

- Flags (Zero, Carry, Negative, Overflow)
- Control bits

5. Pipeline Concept in ARM

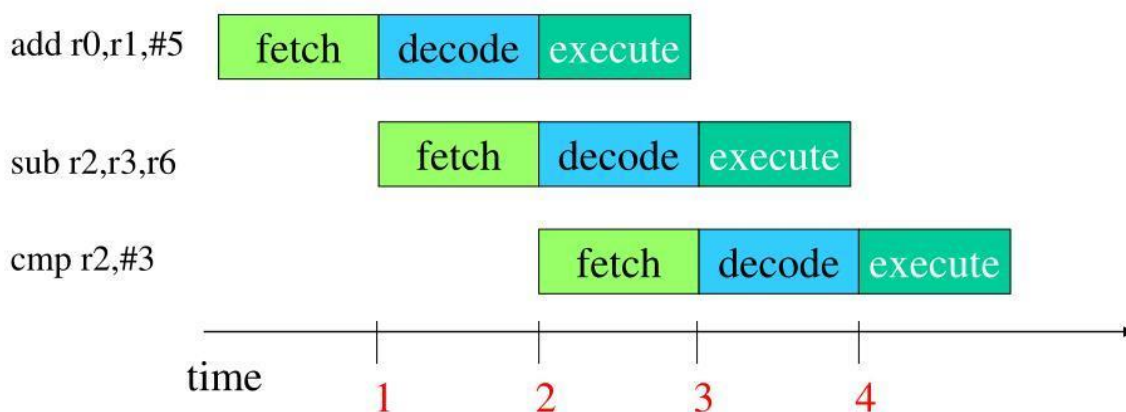
ARM7 Pipeline Model

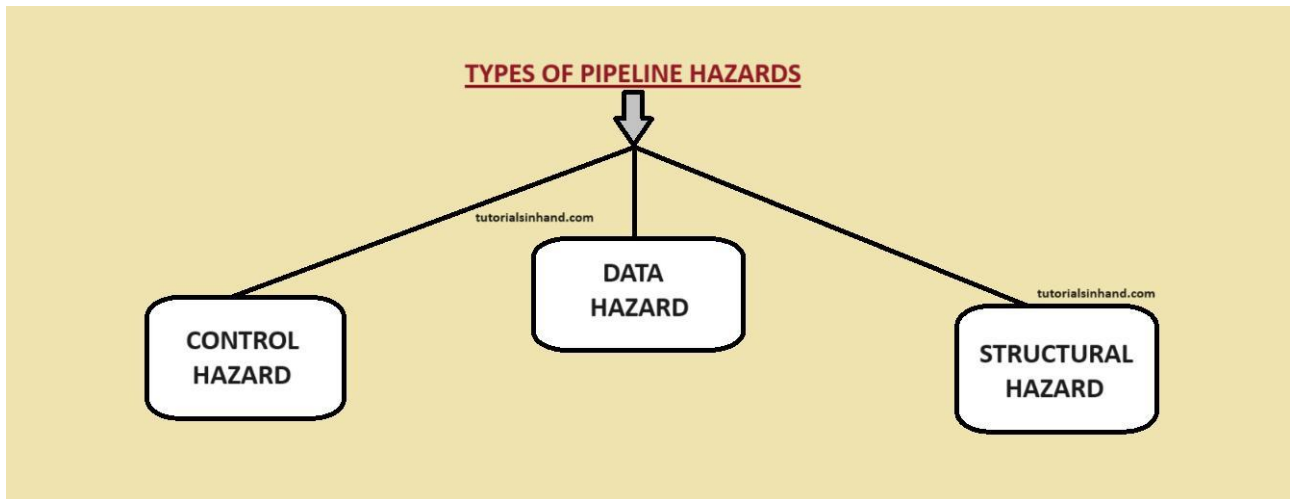
■ ARM7 → standard 3-stage pipelined architecture



*Register write back (WB) is hidden

ARM pipeline execution





Typical 3-stage pipeline:

1. **Fetch** → Instruction fetched from memory
2. **Decode** → Instruction decoded
3. **Execute** → Operation performed

👉 Advantage:

- Multiple instructions processed at once
- Increases throughput

6. Instruction Set Features

- Fixed-length instructions (mostly 32-bit)
- Supports:
 - Data processing (ADD, SUB)
 - Branching
 - Load/Store
- Conditional execution of instructions

7. Addressing Modes

ARM supports various addressing modes:

- Immediate
- Register
- Register indirect
- Indexed addressing

8. Modes of Operation

ARM supports different modes:

- User mode
- Supervisor mode
- Interrupt mode
- System mode

👉 Used for security and task management

9. Interrupt Handling in ARM

- Uses **vector table**
- Supports:
 - IRQ (normal interrupt)
 - FIQ (fast interrupt)

10. Advantages of ARM Processor

- High performance with low power
- Efficient instruction execution
- Compact code size
- Widely supported architecture

11. Applications

- Mobile phones
- Embedded systems
- Robotics
- Automotive systems
- IoT devices
- **Registers in ARM – Detailed Explanation**
- In the ARM Processor, **registers** are small, high-speed storage locations inside the CPU that hold data, addresses, and instructions during execution. ARM uses a **register-based (RISC) architecture**, so most operations are performed directly on registers rather than memory.

Classification of Registers

◆ A. General-Purpose Registers (R0–R12)

- Used for:
 - Arithmetic operations
 - Data storage
 - Intermediate results

B. Special Purpose Registers

1. R13 – Stack Pointer (SP)

- Points to the **top of the stack**
- Used in:
 - Function calls
 - Interrupt handling

2. R14 – Link Register (LR)

- Stores **return address** during function calls
- Helps avoid frequent stack access

👉 Example:

- During a function call, return address is stored in LR

3. R15 – Program Counter (PC)

- Holds address of the **next instruction**
- Automatically updated during execution.

3. Program Status Registers

◆ CPSR (Current Program Status Register)

Stores:

- **Condition flags:**
 - N → Negative
 - Z → Zero
 - C → Carry
 - V → Overflow

- **Control bits:**
 - Interrupt enable/disable
 - Processor mode

◆ **SPSR (Saved Program Status Register)**

- Stores previous CPSR value during interrupts
- Used to restore processor state after ISR

Key Characteristics of ARM Registers

- All registers are **32-bit wide**
- Most instructions operate directly on registers
- Supports **load/store architecture**
- High-speed access compared to memory

7. Advantages

- ⚡ Fast execution
- 🧠 Efficient data handling
- 🔄 Reduced memory access
- 🚀 Better interrupt handling

8. Limitations

- Limited number of registers
- Requires careful usage in complex programs

❖ **Current Program Status Register (CPSR)**

In the ARM Processor, the **Current Program Status Register (CPSR)** is a **32-bit special-purpose register** that holds important information about the **current state of the processor**.

It controls:

- Execution status
- Arithmetic results
- Processor mode
- Interrupt handling

Advantages of CPSR

- Enables conditional execution
- Helps in efficient decision-making
- Controls processor modes
- Supports interrupt handling

10. Limitations

- Requires understanding of flags
- Incorrect flag usage may cause errors

❖ **Pipeline Concept**

In the ARM Processor, **pipelining** is a technique used to **increase instruction execution speed** by overlapping multiple instruction stages. Instead of executing one instruction at a time, ARM processes **several instructions simultaneously in different stages**.

What is Pipelining?

Pipelining divides instruction execution into stages so that:

- While one instruction is executing,
- Another is being decoded,
- And another is being fetched

👉 This improves **throughput (instructions per second)**.

Advantages of Pipelining

- ⚡ Increased instruction throughput
- 🚀 Better CPU utilization
- ⌚ Faster program execution
- 🔄 Parallel processing of instructions

Limitations of Pipelining

- Complex design
- Pipeline hazards reduce efficiency
- Branch instructions may cause delays

Applications

- High-performance computing
- Embedded systems
- Mobile processors
- Real-time systems

❖ Interrupts and Vector Table

◆ 1. What is an Interrupt?

An **interrupt** is a signal that temporarily stops the normal execution of a program (CPU) and transfers control to a special function called an **Interrupt Service Routine (ISR)** to handle urgent tasks.

👉 After servicing the interrupt, the CPU returns to its previous execution.

◆ 2. Why Interrupts are Needed

Interrupts are essential because they allow:

- Immediate response to external/internal events
- Efficient CPU utilization (no continuous polling required)
- Real-time processing in embedded systems

◆ 3. Types of Interrupts (ARM)

◆ A. Based on Source

1. Hardware Interrupts

- Generated by external devices (keyboard, sensors, timers)

2. Software Interrupts (SWI)

- Generated by instructions (e.g., system calls)

◆ B. Based on Priority (ARM specific)

1. FIQ (Fast Interrupt Request)

- Highest priority
- Faster handling (dedicated registers)

2. IRQ (Interrupt Request)

- Normal priority
- General-purpose interrupt handling

◆ 4. Interrupt Handling Mechanism (Step-by-Step)

When an interrupt occurs:

1. Current instruction execution completes
2. CPU saves context:
 - Program Counter (PC)
 - CPSR (Current Program Status Register)

3. CPU switches to **interrupt mode**
4. CPU jumps to a predefined address (from vector table)
5. ISR executes
6. After completion:
 - Context is restored
 - Execution resumes

◆ 5. Interrupt Modes in ARM

ARM supports multiple modes for handling interrupts:

- **User Mode** – Normal program execution
- **FIQ Mode** – For fast interrupts
- **IRQ Mode** – For normal interrupts
- **Supervisor Mode** – For OS/system-level tasks

Each mode has its own **banked registers** to speed up context switching.

◆ What is a Vector Table?

The **Vector Table** is a fixed memory area that contains **addresses of interrupt handlers (ISRs)**.

👉 When an interrupt occurs, the CPU uses this table to find where the corresponding ISR is located.

◆ Example Flow (IRQ Handling)

1. External device triggers IRQ
2. CPU completes current instruction
3. CPU saves context
4. CPU jumps to **0x18 (IRQ vector)**
5. Branch to ISR
6. ISR executes
7. Return using special instruction
8. CPU resumes normal execution

◆ 10. Key Features of ARM Interrupt System

- Fast response using FIQ
- Vector-based handling
- Banked registers (FIQ has more registers → faster execution)
- Supports nested interrupts (advanced ARM cores)

◆ 11. Advantages of Interrupts

- Improves efficiency
- Reduces CPU idle time
- Enables real-time response
- Better handling of asynchronous events

◆ 12. Disadvantages

- Complexity in programming
- Context switching overhead
- Risk of priority inversion
- Debugging becomes difficult